



INSTITUT PASTEUR DE MADAGASCAR

SpécimenManager

Livre de Conception & Audit Technique

Unité d'Entomologie Médicale

Application de gestion des spécimens entomologiques de terrain

Version 1.0 • 29 juillet 2026

Document technique — Architecture, modélisation UML & audit qualité

Sommaire

1. Présentation générale

1.1 Contexte & objectifs — 1.2 Périmètre fonctionnel — 1.3 Glossaire métier

2. Architecture technique

2.1 Pile technologique — 2.2 Architecture en couches — 2.3 Sécurité & contrôle d'accès

3. Modèle de données

3.1 Hiérarchie du domaine — 3.2 Diagramme de classes (cœur) — 3.3 Module Laboratoire — 3.4 Référentiels & énumérations

4. Diagrammes de comportement

4.1 Cas d'utilisation & matrice RBAC — 4.2 Séquence : authentification — 4.3 Séquence : saisie spécimen — 4.4 Séquence : workflow labo — 4.5 Diagrammes d'états

5. Audit qualité — bugs & améliorations

5.1 Méthode — 5.2 Anomalies backend — 5.3 Anomalies frontend — 5.4 Améliorations transverses

6. Feuille de route de finalisation

Synthèse en un coup d'œil

35

ENTITÉS DE
DONNÉES

24

CONTRÔLEURS API

44

PAGES REACT

5

RÔLES
UTILISATEURS

17

MIGRATIONS DB

Objet du document. Ce livre décrit la conception de SpécimenManager (modélisation UML complète) et présente un audit technique identifiant les anomalies probables et les améliorations recommandées avant la mise en production finale.

1. Présentation générale

1.1 Contexte & objectifs

SpécimenManager est une application web développée pour l'**Unité d'Entomologie Médicale de l'Institut Pasteur de Madagascar**. Elle assure la traçabilité complète des spécimens entomologiques (moustiques, tiques, puces et autres arthropodes) collectés lors des missions de terrain, depuis la collecte jusqu'aux analyses de laboratoire (biologie moléculaire).

L'application répond à trois besoins :

- **Saisie structurée & normalisée** — toute donnée scientifique structurante référence un dictionnaire (taxonomie, méthode de collecte, solution de conservation...), interdisant le texte libre incohérent.
- **Chaîne de traçabilité** — un spécimen est toujours rattaché à sa méthode de collecte, elle-même rattachée à une localité, une mission, un projet.
- **Analyses de laboratoire** — un module complet documente les 9 types de manipulations (identification morphologique, extraction ADN/ARN, PCR, qPCR, PCR nichée, séquençage, microscopie...) avec validation scientifique et audit immuable.

1.2 Périmètre fonctionnel

Domaine	Fonctions couvertes
Terrain	Projets, missions, localités (GPS), méthodes de collecte, hôtes, containers (plaques 96 puits / boîtes 81 tubes)
Spécimens	Moustiques, tiques, puces, autres — saisie unitaire, import Excel, positionnement en plaque
Laboratoire	9 modules de manipulation, pools d'analyse, pathogènes cibles, upload de gels/fichiers de séquençage
Dictionnaires	Taxonomies hiérarchiques, référentiels simples, activation/désactivation, historique d'audit
Transverse	Cartographie Leaflet, tableau de bord, recherche globale, notifications temps réel (SSE), gestion des utilisateurs

1.3 Glossaire métier

Terme	Définition
Projet	Programme de recherche cadre, porteur de missions.
Mission	Campagne de terrain rattachée à un projet, avec ordre de mission et chef de mission.
Localité	Site géolocalisé (région / district / commune / fokontany) visité pendant une mission.
Méthode de collecte	Instance opérationnelle d'un type de piège (CDC-LT, BG-Sentinel, capture humaine...) posée sur une localité. <i>Pivot central : tout spécimen y est rattaché.</i>
Hôte	Animal sur lequel des tiques/puces sont prélevées, rattaché à une méthode de collecte.
Pool	Regroupement de plusieurs spécimens analysés ensemble (broyage groupé).
Manipulation	Opération de laboratoire (une par type) portant sur un spécimen individuel ou un pool.
Container	Support physique de conservation : plaque 96 puits ou boîte 81 tubes.

2. Architecture technique

2.1 Pile technologique

Couche	Technologies
Frontend	React 19, Vite 5, React Router v7, TanStack Query v5, Zustand, Tailwind CSS v3, Leaflet / react-leaflet, Recharts, Axios
Backend	Node.js, Express 4, Prisma 5 (ORM), Zod v4 (validation), JWT (auth), Helmet, express-rate-limit, Multer (upload), ExcelJS (import/export)
Base de données	PostgreSQL (port 5435, conteneurisée via Docker Compose)
Temps réel	Server-Sent Events (SSE) — notifications, présence, changement de permissions
Déploiement	NAS Synology (Docker), accès distant via Tailscale VPN, reverse-proxy nginx

2.2 Architecture en couches

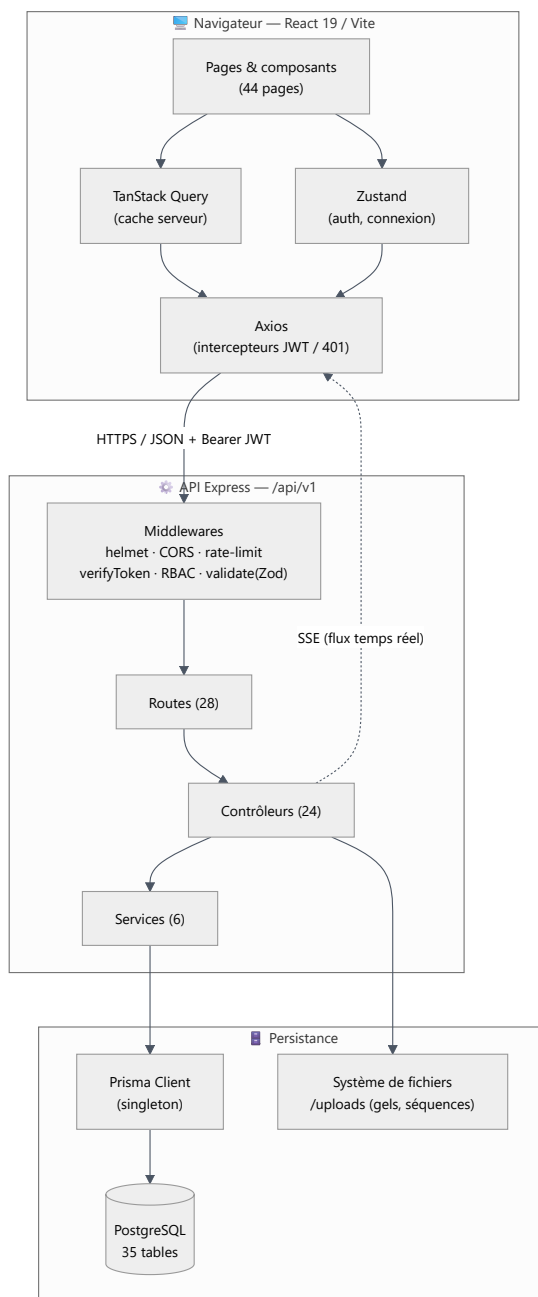


Figure 2.1 — Architecture applicative en couches (client → API → persistance).

Le backend applique une chaîne de middlewares homogène : `helmet` (en-têtes de sécurité) → `CORS` restreint à `CLIENT_URL` → journalisation (avec masquage des `?token=`) → authentification JWT → garde de rôle → validation Zod → contrôleur. Un gestionnaire d'erreurs global traduit les codes Prisma (P2002, P2003, P2025...) et les `ZodError` en réponses HTTP cohérentes.

2.3 Sécurité & contrôle d'accès

Trois mécanismes se superposent :

- **Hiérarchie de rôles** — `admin (5) > superviseur (4) > chercheur (3) > technicien (2) > lecteur (1)`. Deux gardes : `requireRole(...)` (correspondance exacte) et `requireMinRole(r)` (niveau $\geq$).
- **Cloisonnement par projet** — les rôles non-privilegiés ne voient que les projets dont ils sont *membres* (`MembreProjet`). Admin et superviseur ont un accès global (*bypass*).
- **Accès par type de spécimen** — chaque utilisateur possède un tableau `specimensAutorises` ; `checkSpecimenAccess(type)` revérifie l'état en base (cache 60 s) pour refléter immédiatement les changements de l'admin.

Contrôle	Mise en œuvre	Statut
Mots de passe	bcrypt (coût 10)	OK
Jeton de session	JWT signé, expiration 8 h	OK
Force brute login	rate-limit 5 essais / 15 min par IP	OK
En-têtes HTTP	Helmet activé	OK
Secrets	<code>.env</code> hors dépôt, pas de fallback JWT	OK
Révocation de session	non enforced hors routes spécimen	À CORRIGER
Mot de passe admin seed	<code>Admin1234!</code> par défaut	À CHANGER

Détail des deux derniers points au chapitre 5 (anomalies B2 et amélioration sécurité).

3. Modèle de données

3.1 Hiérarchie du domaine

Le cœur métier suit une chaîne de containment stricte. Un spécimen n'est *jamais* rattaché directement à une localité ou une mission : toujours via une méthode de collecte.

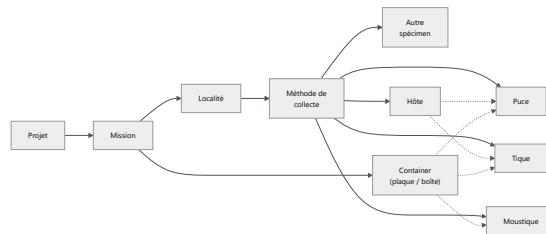


Figure 3.1 — Chaîne de containment. Trait plein = composition (cascade à la suppression) ; pointillé = référence optionnelle.

3.2 Diagramme de classes — cœur du domaine

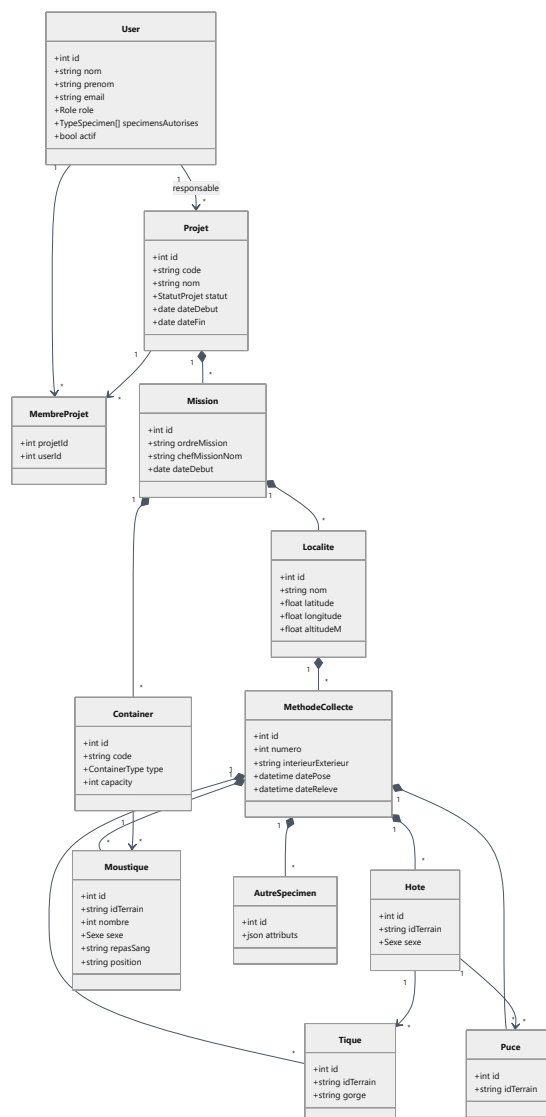


Figure 3.2 — Diagramme de classes du domaine terrain (attributs principaux ; les liens vers les référentiels sont détaillés en 3.4).

3.3 Module Laboratoire

Une table pivot `ManipulationLabo` porte les attributs communs (opérateur, dates, statut de validation) et se spécialise via une relation 1–1 vers l'un des 9 modules selon `typeManipulation`. Chaque manipulation cible **soit** un spécimen individuel **soit** un pool (contrainte applicative). Les `ManipulationEvent` forment un journal immuable (audit scientifique).

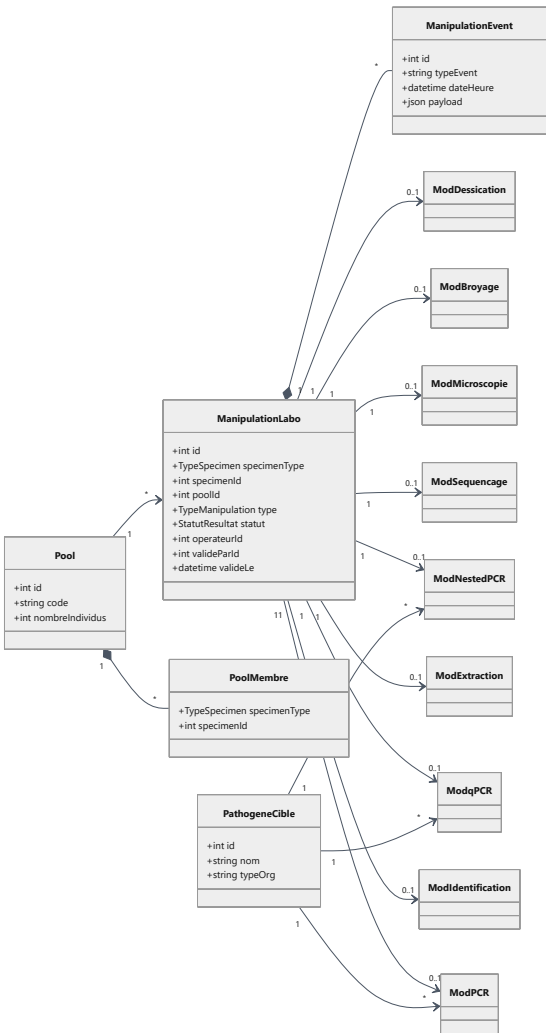


Figure 3.3 — Module Laboratoire : héritage par composition (table-per-type) autour de la table pivot `ManipulationLabo`.

3.4 Référentiels & énumérations

Les dictionnaires garantissent la normalisation des données. Tous partagent le motif `actif / createdBy / updatedBy` pour l'activation et l'audit.

Référentiel	Rôle	Utilisé par
TaxonomieSpecimen	Arbre taxonomique (ordre → espèce)	tous les spécimens
TaxonomieHote	Arbre taxonomique des hôtes	Hôte
TypeMethodeCollecte	Types de piège (CDC-LT, BG...)	MéthodeCollecte
SolutionConservation	Éthanol 70 %, RNAlater...	tous les spécimens
TypeEnvironnement / TypeHabitat	Contexte écologique	MéthodeCollecte

TypeAutreSpecimen	Phlébotomes, Culicoïdes...	AutreSpecimen
PathogeneCible	Agent recherché (PCR/qPCR)	modules moléculaires

Énumération	Valeurs
Role	admin · superviseur · chercheur · technicien · lecteur
StatutProjet	actif · termine · suspendu
StatutResultat	brut · valide · invalide
TypeManipulation	identification_morpho · broyage_pool · dessication · extraction · amplification_pcr · qpcr · nested_pcr · sequencage · microscopie
ContainerType	PLAQUE (96 puits) · BOITE (81 tubes)
Sexe	M · F · inconnu

4. Diagrammes de comportement

4.1 Cas d'utilisation & matrice RBAC

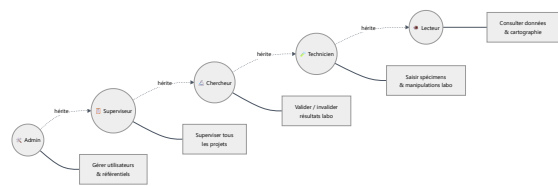


Figure 4.1 — Cas d'utilisation par rôle (les rôles supérieurs héritent des capacités des rôles inférieurs).

Action	Lecteur	Technicien	Chercheur	Superviseur	Admin
Consulter (spécimens, cartes, tableaux)	✓	✓	✓	✓	✓
Créer / éditer spécimens & manipulations	—	✓	✓	✓	✓
Valider / invalider résultats labo	—	—	✓	✓	✓
Accès à tous les projets (bypass cloisonnement)	—	—	—	✓	✓
Gérer utilisateurs / activer comptes	—	—	—	—	✓
Modifier un résultat déjà validé	—	—	—	—	✓

4.2 Séquence — authentification

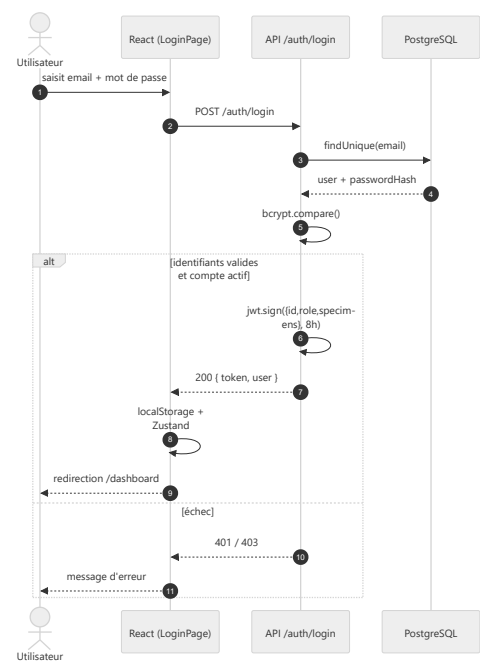


Figure 4.2 — Séquence de connexion (JWT).

4.3 Séquence — saisie d'un spécimen de terrain

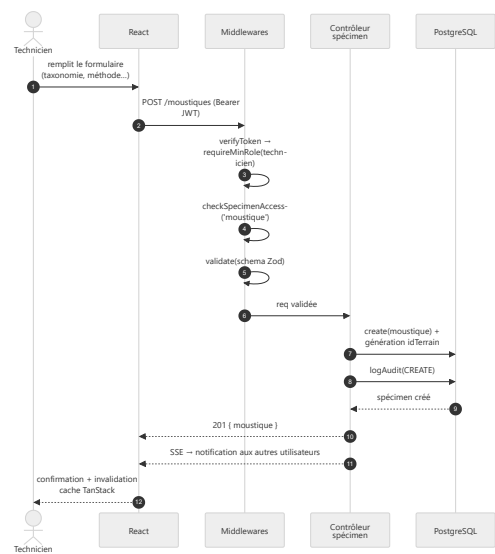


Figure 4.3 — Création d'un spécimen : gardes de sécurité en cascade puis persistance + audit + notification temps réel.

4.4 Séquence — workflow de manipulation de laboratoire

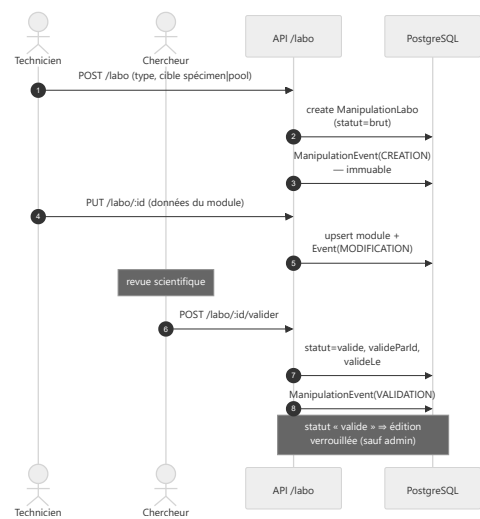


Figure 4.4 — Cycle de vie d'une manipulation : saisie (technicien) → validation (chercheur) → verrouillage.

4.5 Diagrammes d'états

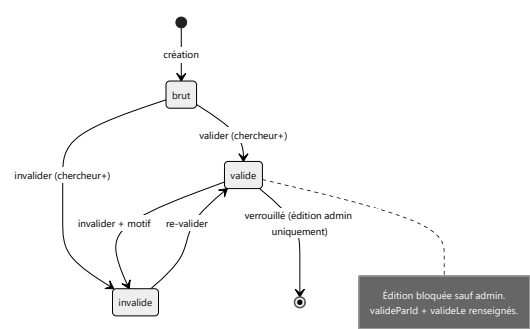


Figure 4.5 — États d'une ManipulationLabo (StatutResultat).

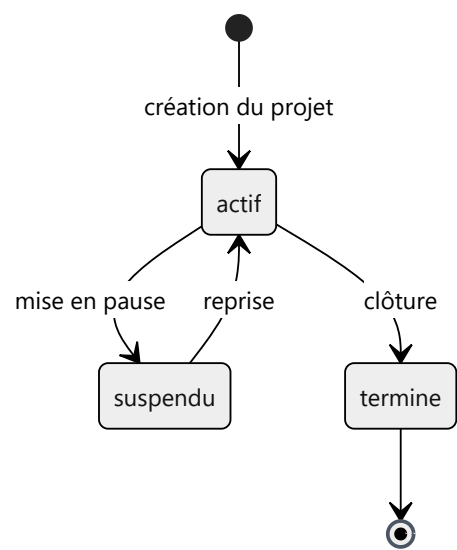


Figure 4.6 — États d'un Projet (StatutProjet).

5. Audit qualité — bugs & améliorations

5.1 Méthode

L'audit résulte d'une lecture ciblée du schéma Prisma, des middlewares de sécurité, des contrôleurs critiques (auth, labo, notifications) et des couches frontend d'authentification. Chaque anomalie est classée par sévérité et accompagnée d'un correctif concret.

MAJEUR impact fonctionnel/sécurité — à traiter avant production. **MOYEN** dégrade l'expérience ou l'intégrité. **MINEUR** dette technique / robustesse.

5.2 Anomalies — Backend

B1 **MAJEUR** — Fichiers de laboratoire jamais supprimés du disque

labo.controller.js > deleteManipulation

Le chemin de suppression est reconstruit par `p.replace(/^labo\/[^\/]+\//, '')`, qui retire le sous-dossier (`images/`, `sequencage/`). Or `UPLOADS_ROOT` se termine déjà par `/labo` : la jointure vise donc `uploads/labo/<fichier>` au lieu de `uploads/labo/images/<fichier>`. `fs.existsSync()` renvoie faux et le fichier n'est jamais effacé.

Conséquence : accumulation de fichiers orphelins (images de gels, fichiers `.fastq` jusqu'à 500 Mo) à chaque suppression de manipulation.

Correctif : ne retirer que le préfixe `labo/` : `p.replace(/^labo\/\//, '')`.

B2 **MAJEUR** — Absence de révocation de session

auth.middleware.js > verifyToken

`verifyToken` se fie uniquement à la signature du JWT (durée 8 h) sans vérifier l'état du compte. Désactiver (`actif=false`), rétrograder ou supprimer un utilisateur ne coupe pas son accès aux routes non-spécimen (projets, missions, labo, dashboard) jusqu'à l'expiration du token. `kickUser` ne ferme que le flux SSE. Seul `checkSpecimenAccess` vérifie `actif`.

Correctif : vérifier `actif` + `role` en base dans `verifyToken` (avec cache court partagé), ou introduire un `tokenVersion` incrémenté à chaque changement sensible et comparé au claim du JWT.

B3 **MOYEN** — État « lu » des notifications global au lieu de par-utilisateur

notifications.controller.js — réutilise `audit_logs.isRead`

`isRead` est un booléen unique porté par la ligne d'audit, partagé par tous les utilisateurs. Lorsqu'un utilisateur marque une notification comme lue, elle disparaît pour *tout le monde*, et le `unreadCount` individuel est faussé.

Correctif : introduire une table de jonction `NotificationRead(userId, auditLogId)`, ou générer une notification par destinataire.

B4 **MOYEN** — Mise à jour partielle qui écrase les données scientifiques

labo.controller.js > updateManipulation / buildModuleData

`buildModuleData` renvoie *tous* les champs du module (les absents deviennent `null`), et `upsert.update: modData` réécrit l'ensemble. Un `PUT` partiel (par ex. modifier seulement `notes`) remet à `null` toutes les valeurs du module (Ct, concentrations, séquences...).

Correctif : ne conserver dans `modData` que les clés effectivement présentes dans `req.body` avant l'`upsert`.

B5 MOYEN — « Autres spécimens » inaccessibles aux rôles non privilégiés

auth.controller.js > SPECIMENS_VALIDES

`SPECIMENS_VALIDES = ['moustique', 'tique', 'puce']` et `updateSpecimenAccess` rejette la valeur `'autre'`. Or `checkSpecimenAccess('autre')` exige que `'autre'` figure dans `specimensAutorises`. Un chercheur/technicien/lecteur reçoit donc toujours 403 sur les autres spécimens, et l'entrée n'apparaît jamais dans sa barre latérale. Incohérence introduite lors de l'ajout tardif du type `autre`.

Correctif : ajouter `'autre'` à `SPECIMENS_VALIDES` (et, si souhaité, au défaut de `specimensAutorises`).

B6 MINEUR — Relations polymorphes sans intégrité référentielle

schema.prisma > PoolMembre, ManipulationLabo

Le ciblage d'un spécimen (`specimenType` + `specimenId`) n'est pas une clé étrangère : supprimer un spécimen laisse des `PoolMembre` et manipulations pendantes. La contrainte « spécimen $\oplus$ pool » n'existe qu'au niveau applicatif.

Correctif : nettoyage applicatif à la suppression d'un spécimen (ou contrainte `CHECK` + trigger), et documenter la règle d'exclusivité.

B7 MINEUR — Code mort & double validation

Plusieurs contrôleurs (`register`, `createUser`, `updateUser` ...) refont une validation manuelle alors que les routes appliquent déjà `validate(schema.*)` (Zod). Par ailleurs `upload.middleware.js` exporte `uploadGel` / `uploadFichier` inutilisés (`labo.routes.js` redéfinit son propre `makeUpload`).

Correctif : supprimer les validations redondantes et le code d'upload mort.

5.3 Anomalies — Frontend

F1 MOYEN — Rôle affiché figé après modification par l'admin

`store/authStore.js`, SSE `permissions_changed`

`user.role` provient du token / `localStorage`. Le SSE ne pousse que les `specimensAutorises` (via `updateUser`), pas le rôle. Un utilisateur promu/rétrogradé conserve donc une interface (barre latérale, boutons d'action) obsolète jusqu'à reconnexion.

Correctif : pousser aussi le rôle via SSE, ou rafraîchir périodiquement l'utilisateur depuis `/auth/me`.

F2 MINEUR — Duplication de la logique de rôles back/front

La hiérarchie des rôles et la liste des types de spécimens sont dupliquées entre le backend et `frontend/src/lib/roles.js`. Cette dérive a déjà causé un bug (blocage du *superviseur*, corrigé récemment).

Correctif : exposer la configuration RBAC via un endpoint unique, ou centraliser dans un module partagé.

F3 MINEUR — Redirection 401 non protégée contre les rafales

`api/axios.js`

Sur 401, l'intercepteur planifie `window.location.href = '/login'` après 1200 ms. Plusieurs requêtes échouant simultanément peuvent empiler redirections et toasts.

Correctif : ajouter un garde (drapeau `isRedirecting`) pour ne déclencher la redirection qu'une seule fois.

5.4 Améliorations transverses

Axe	Recommandation	Priorité
Tests	Aucun framework (backend : stub ; frontend : néant). Ajouter Vitest + Supertest côté API et React Testing Library côté UI ; câbler le <code>smoke-test.js</code> existant.	HAUTE
CI/CD	<code>.github/</code> vide. Workflow lint + smoke-test + <code>prisma migrate diff</code> à chaque push.	HAUTE
Sécurité	Forcer le changement du mot de passe admin seed au 1 ^{er} login ; envisager des URLs signées pour les fichiers d'upload.	MOYENNE
Scalabilité	Cache de permissions in-memory par processus : basculer sur Redis si déploiement multi-instances (PM2 cluster).	MOYENNE
Observabilité	Remplacer <code>console.log</code> par un logger structuré (pino/winston) avec niveaux et rotation.	BASSE
Données	Valider le champ libre <code>AutreSpecimen.attributes</code> (JSON) via un schéma Zod par type.	BASSE
Dépôt	Sortir les binaires <code>presentation/*.pptx docx</code> (~8–10 Mo) du dépôt Git (LFS ou stockage externe).	BASSE
i18n	Compléter la traduction (<code>useT</code> partiellement déployé).	BASSE

6. Feuille de route de finalisation

Séquence recommandée pour clôturer la version 1.0, du plus critique au confort.

Sprint 1 — Correctifs bloquants (avant production)

1. **B1** — corriger le chemin de suppression des fichiers labo.
2. **B2** — enforcer la révocation de session (vérification `actif` /rôle).
3. **B5** — rendre les « autres spécimens » attribuables aux rôles standards.
4. Changer le mot de passe admin par défaut.

Sprint 2 — Intégrité & expérience

1. **B3** — notifications « lu » par utilisateur.
2. **B4** — mises à jour partielles non destructrices en labo.
3. **F1** — synchronisation du rôle en temps réel.
4. **F3** — garde de redirection 401.

Sprint 3 — Fiabilisation & industrialisation

1. Mise en place des tests (API + UI) et de la CI GitHub Actions.
2. Nettoyage du code mort (**B7**) et centralisation RBAC (**F2**).
3. Logger structuré, nettoyage du dépôt, complétion i18n.
4. **B6** — intégrité des relations polymorphes.

Conclusion. SpécimenManager présente une architecture saine et cohérente : modèle de données rigoureux, sécurité multi-niveaux (JWT, RBAC hiérarchique, cloisonnement par projet), et un module laboratoire abouti. Les anomalies identifiées sont ciblées et corrigeables rapidement ; aucune ne remet en cause la conception d'ensemble. Le principal chantier de maturité reste l'**automatisation des tests et de l'intégration continue**, indispensable pour sécuriser les évolutions futures.

